

# SequelSpeak: Base Model Prompt Optimization

## 🎯 Major Updates for Base Model Optimization

### 1. Completion-Style Prompting

- **Before:** Instruction-following format with system prompts
- **After:** Pattern completion format that base models excel at
- Uses direct examples without complex instructions

### 2. Base Model-Specific Architecture

```
model_configs = {  
    "openai_base": "code-davinci-002",      # Best for SQL  
    "codegen": "codegen-350M-mono",        # Fast & lightweight  
    "starcode": "starcode-base",           # Strong code understanding  
    "text_davinci": "text-davinci-003"     # General purpose  
}
```

### 3. Optimized Prompt Structure

# Text-to-SQL Examples - Convert natural language to PostgreSQL queries

Database Schema Context: {schema\_context}

Examples:

Natural Language: Show me all customers from California

SQL: SELECT \* FROM customers WHERE state = 'California';

Natural Language: {natural\_question}

SQL:

### 4. Key Features for Base Models

-  **Stop Tokens:** [";", "\n\n", "Natural Language:"] prevent over-generation
-  **Post-Processing:** Extracts clean SQL from completions
-  **Pattern Recognition:** Consistent formatting for better learning
-  **Simplified Schema:** Optimized format for base model understanding

### 5. Base Model-Specific Scoring Rubric

Added specialized criteria:

- **Completion Quality (30%)** - Pattern following effectiveness
- **SQL Syntax Accuracy (25%)** - PostgreSQL correctness
- **Semantic Understanding (25%)** - Intent interpretation
- **Schema Adherence (20%)** - Correct schema usage

## 6. Model Comparison Framework

Comprehensive evaluation system for:

- **OpenAI Base Models:** code-davinci-002, text-davinci-003
- **Hugging Face Models:** CodeGen, StarCoder, etc.
- **Performance Metrics:** Speed, accuracy, cost-effectiveness



### Ready for Integration

```
prompt_system = SequelSpeakPromptTemplates({
    "model_name": "code-davinci-002",
    "api_key": "your-key",
    "temperature": 0.1,
    "max_tokens": 200,
    "stop_tokens": [";", "\n\n", "Natural Language:"]
})

sql_query = prompt_system.generate_sql_with_base_model(
    "Show me top customers",
    schema_context
)
```



### Base Model Advantages

1. **Cost Effective:** Generally cheaper than instruction-tuned models
2. **Fast Inference:** Optimized for completion tasks
3. **Specialized:** Code-focused models excel at SQL generation
4. **Consistent:** More predictable outputs with proper prompting
5. **Customizable:** Easier to fine-tune for specific domains



### Recommended Models for SequelSpeak

| Model                   | Provider   | Strengths                 | Use Case        |
|-------------------------|------------|---------------------------|-----------------|
| <b>code-davinci-002</b> | OpenAI     | Best SQL generation       | Production      |
| <b>CodeGen-350M</b>     | Salesforce | Fast, lightweight         | Development     |
| <b>StarCoder-base</b>   | BigCode    | Strong code understanding | Complex queries |
| <b>text-davinci-003</b> | OpenAI     | High quality completion   | General purpose |

The system is now **fully optimized for base models** and ready for integration into your SequelSpeak RAG pipeline! 🎉